

# คู่มือปฏิบัติการฉบับสมบูรณ์: เครื่องมือทดสอบและประเมิน WAF ในงานจริง

## WAF Testing & Evaluation Tools Handbook (Security Engineering Standard)

เจาะลึก 5 เครื่องมือมาตรฐานสากล | วัดประสิทธิภาพ | ประโยชน์ที่ได้รับ | ชุดคำสั่งพร้อมรัน | อธิบายพารามิเตอร์ตัวอย่างละเอียด

### 1. สถาปัตยกรรมและกรอบการประเมิน WAF ในระดับองค์กร

ในสายงานวิศวกรรมความมั่นคงปลอดภัยไซเบอร์ (Security Engineering) การประเมิน Web Application Firewall (WAF) มีเป้าหมายเพื่อตอบคำถาม 4 ประเด็นหลักอย่างเป็นรูปธรรม:

- 1. Detection Efficacy (ความแม่นยำในการสกัดกั้น):** กฎของ WAF สามารถตรวจจับและบล็อกการโจมตีมาตรฐานสากล (เช่น OWASP Top 10) ได้จริงกี่เปอร์เซ็นต์
- 2. Evasion Resistance (ความทนทานต่อการหลบเลี่ยง):** WAF มีความสามารถในการถอดรหัส (Normalize) รูปแบบการโจมตีที่ผ่านการ Obfuscate, Chunked, หรือ Double Encoding ได้ดีเพียงใด
- 3. False Positive Rate (อัตราการบล็อกผู้ใช้ปกติผิดพลาด):** คำขอปกติทางธุรกิจ (เช่น ข้อความภาษาไทย, อักขระพิเศษในฟอร์ม, ข้อมูล JSON/XML ขนาดใหญ่) ถูก WAF สั่งบล็อกผิดพลาดหรือไม่
- 4. Performance Overhead (ผลกระทบต่อความเร็ว):** การตรวจวิเคราะห์แบบ Deep Packet Inspection ทำให้ Latency เพิ่มขึ้นกี่มิลลิวินาที (P95 Overhead) และรองรับ Throughput (RPS) ได้สูงสุดเท่าไร

### 2. แผนที่เลือกใช้เครื่องมือให้ตรงกับโจทย์ (TOOL SELECTION MATRIX)

| เครื่องมือ   | โจทย์และเป้าหมายการทดสอบ                                                    | ผลลัพธ์หลักที่ได้รับ                                    | โปรโตคอล/ระดับ     | ความยากในการติดตั้ง       |
|--------------|-----------------------------------------------------------------------------|---------------------------------------------------------|--------------------|---------------------------|
| 1. GoTestWAF | วัดเกรด Benchmark และหาช่องโหว่ Bypass จากเทคนิค Encoders ต่างๆ             | เกรด A+ ถึง F, % Bypass Rate, PDF/CSV Report            | L7 HTTP / API      | ง่าย (Single Binary)      |
| 2. OWASP FTW | ตรวจสอบความถูกต้องของกฎ ModSecurity CRS ที่ละ Rule ID (Regression)          | Pass/Fail ราย Rule, HTTP Status Verification            | L7 ModSec Engine   | ปานกลาง (Python/pytest)   |
| 3. Wafw00f   | สำรวจ ยืนยันชื่อ WAF และตรวจสอบว่าทราฟฟิกวิ่งผ่าน WAF จริงหรือไม่           | ชื่อ WAF, การตอบสนองของ Filter เบื้องต้น                | L7 Fingerprinting  | ง่ายมาก (Python CLI)      |
| 4. OWASP ZAP | ตรวจสอบว่า WAF สามารถกักบัง (Shield) ช่องโหว่ของเว็บแอปพลิเคชันได้จริงไหม   | เปรียบเทียบ Vulnerability ระหว่าง Direct vs Through WAF | DAST Full Web App  | ปานกลาง (GUI/Docker)      |
| 5. k6        | วัดความหน่วงส่วนเพิ่ม (Added Latency P95) และพิกัด Throughput เมื่อเปิด WAF | Response Time (ms), Max RPS, CPU Bottleneck             | Performance / Load | ง่าย (Single Binary / JS) |

ลำดับการทำงานที่แนะนำ (Recommended Workflow): เริ่มต้นด้วย Wafw00f เพื่อยืนยันว่า WAF ดักจับทราฟฟิกจริง → รัน GoTestWAF เพื่อหาคะแนน Benchmark และอัตรา Bypass รวม → ใช้ OWASP FTW ในการปรับกฎที่ละข้อ → และวัดความเร็วด้วย k6

### 3. เครื่องมือที่ 1: GOTESTWAF (WALLARM)

#### GoTestWAF — เครื่องมือ Benchmark วัดความเหนียวและคำนวณ Bypass Rate อัตโนมัติ

ประเภท: Open-Source CLI (Go) | เหมาะสำหรับ: SecOps, QA Engineer, ผู้ประเมินความปลอดภัย WAF

**เครื่องมือนี้ทำอะไร:** เป็นโปรแกรมส่ง HTTP Requests อัตโนมัติที่บรรจุ Payload ทดสอบการโจมตีตามมาตรฐาน OWASP Top 10 (SQLi, XSS, RCE, LFI, SSRF ฯลฯ) และ API Security โดยจุดเด่นคือการสลับตำแหน่งวาง Payload (URLParam, Header, HTMLForm, JSONBody) ร่วมกับ Encoders หลากหลายประเภท (Base64, URL, Double URL, Unicode, Hex)

**ช่วยอะไรในการทดสอบ WAF:**

- คำนวณคะแนนประสิทธิภาพ (Security Score) และตัดเกรดออกมาเป็น A+, B, C, D, F ทำให้เปรียบเทียบระหว่าง WAF แต่ละเจ้าหรือแต่ละเวอร์ชันได้ชัดเจน
- คำนวณอัตรา Bypass Rate (%) ในแต่ละหมวดหมู่ ทำให้ทราบทันทีว่า WAF ของเรารองรับการโจมตีแบบใด
- ตรวจสอบอัตรา False Positive ต่อข้อความปกติภาษาอังกฤษ

#### ชุดคำสั่งใช้งานจริง (Full Command Examples)

1. คำสั่งมาตรฐานสำหรับทดสอบเป้าหมายแบบครบวงจร (บันทึกรายงาน PDF และ CSV):

```
cd ~/gotestwaf
./gotestwaf --url=http://localhost:8080 --configPath=config.yaml --testCasesPath=testcases --
reportPath=reports --reportFormat=pdf,csv --passStatusCodes=200,404,405 --blockStatusCodes=403 --
skipWAFBlockCheck --workers=5 --sendDelay=400
```

## 2. คำสั่งทดสอบเฉพาะหมวดหมู่ (Targeted Scan):

```
# ทดสอบเฉพาะหมวด OWASP SQL Injection
./gotestwaf --url=http://localhost:8080 --testCase=sql-injection --skipWAFBlockCheck

# ทดสอบเฉพาะชุดข้อความปกติ (ตรวจสอบว่าล็อกคำภาษาอังกฤษทั่วไปหรือไม่)
./gotestwaf --url=http://localhost:8080 --testSet=false-pos --skipWAFBlockCheck
```

## ตารางอธิบายพารามิเตอร์ของ GoTestWAF อย่างละเอียด

| พารามิเตอร์ (Flag)               | ชนิดข้อมูล     | ความหมายและการทำงานเชิงลึก                                                                              | ตัวอย่างการใช้งานจริง                      |
|----------------------------------|----------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>--url</code>               | String (URL)   | กำหนด URL ของระบบเป้าหมายที่มี WAF ขวางอยู่หน้าบ้าน                                                     | <code>--url=http://127.0.0.1:8080</code>   |
| <code>--configPath</code>        | File Path      | ไฟล์คอนฟิกกำหนด Headers ที่ส่งไปกับ Request เช่น User-Agent หรือ Accept-Encoding                        | <code>--configPath=config.yaml</code>      |
| <code>--testCasesPath</code>     | Folder Path    | โฟลเดอร์บรรจุไฟล์ Test Cases ในรูปแบบ YAML (มี owasp, false-pos, community)                             | <code>--testCasesPath=testcases</code>     |
| <code>--reportPath</code>        | Folder Path    | ไดเรกทอรีปลายทางที่ให้โปรแกรมบันทึกไฟล์สรุปผล                                                           | <code>--reportPath=reports</code>          |
| <code>--reportFormat</code>      | String List    | กำหนดรูปแบบรายงานที่ต้องการส่งออก (เลือกได้: pdf, csv, html, json)                                      | <code>--reportFormat=pdf, csv</code>       |
| <code>--passStatusCodes</code>   | Integer List   | รหัส HTTP ที่ระบบมองว่า "ยอมให้ผ่าน" โดยค่าเริ่มต้นคือ 200, 404 (แนะนำใส่ 405 เพิ่มหาก Backend ตอบ 405) | <code>--passStatusCodes=200,404,405</code> |
| <code>--blockStatusCodes</code>  | Integer List   | รหัส HTTP ที่ระบบมองว่า "WAF สกัดกันสำเร็จ" (ค่ามาตรฐานคือ 403)                                         | <code>--blockStatusCodes=403</code>        |
| <code>--skipWAFBlockCheck</code> | Boolean (Flag) | ข้ามการตรวจสอบ WAF เริ่มต้น (ป้องกันโปรแกรมตัดจบการทำงานหากตรวจไม่พบ WAF ยี่ห้อยัง)                     | <code>--skipWAFBlockCheck</code>           |
| <code>--workers</code>           | Integer        | จำนวน Concurrent Threads ที่ส่งคำขอพร้อมกัน (แนะนำ 3-5 เพื่อไม่ให้เว็บล่ม)                              | <code>--workers=5</code>                   |
| <code>--sendDelay</code>         | Integer (ms)   | ระยะเวลาหน่วงเวลาระหว่าง Request ป้องกันการติด Rate Limiting หรือเครื่องหน่วง                           | <code>--sendDelay=400</code>               |
| <code>--testSet</code>           | String         | เลือกยี่ห้อเฉพาะหมวดหมู่ใหญ่ เช่น owasp, owasp-api, false-pos, community                                | <code>--testSet=owasp</code>               |
| <code>--testCase</code>          | String         | เลือกยี่ห้อเฉพาะเคสย่อย เช่น sql-injection, xss-scripting, nosql-injection                              | <code>--testCase=xss-scripting</code>      |

### ข้อผิดพลาดที่พบบ่อย (Common Pitfalls):

1. sudo: ./gotestwaf: command not found → เกิดจากไม่ได้ใช้คำสั่ง `cd ~/gotestwaf` ก่อนรัน หรือไม่ได้ระบุ Path เต็ม
2. คำขอขึ้นสถานะ unresolved จำนวนมาก → เกิดจากเซิร์ฟเวอร์ตอบกลับรหัส HTTP 405 แก้ได้โดยเพิ่ม `--passStatusCodes=200,404,405`

## 4. เครื่องมือที่ 2: OWASP FTW (FRAMEWORK FOR TESTING WAFS)

### OWASP FTW — เครื่องมือ Regression Testing สำหรับตรวจสอบกฎ ModSecurity CRS ที่ละข้อ

ประเภท: Python Test Engine / pytest Plugin | เหมาะสำหรับ: WAF Rule Developer, Security Engineer

**เครื่องมือนี้ทำอะไร:** เป็น Test Engine มาตรฐานของโปรเจกต์ OWASP Core Rule Set (CRS) พัฒนาด้วย Python ใช้ส่ง HTTP Request สำเร็จรูปที่เขียนในไฟล์ YAML เพื่อตรวจสอบว่าแต่ละกฎของ WAF ตอบสนองได้ถูกต้องตามเงื่อนไขที่กำหนดไว้หรือไม่

#### ช่วยอะไรในการทดสอบ WAF:

- ตรวจสอบราย Rule ID (เช่น Rule 942100 ตรวจสอบ SQLi) ว่าเมื่อมี Payload เข้ามาแล้วติดกฎข้อนั้นจริงหรือไม่

- ป้องกันการถอยหลัง (Prevent Regression): เมื่อเราเข้าไปจนกฏข้อหนึ่ง มันใจได้ว่าจะไม่ส่งผลกระทบต่อกฏข้ออื่นๆ ในระบบ
- ใช้ยืนยันการทำงานของระดับ Paranoia Level (PL 1 ถึง PL 4) ของ ModSecurity CRS

#### ชุดคำสั่งใช้งานจริง (Full Command Examples)

```
# 1. ติดตั้ง FTW และ pytest บนสภาพแวดล้อม Python
pip install ftw pytest

# 2. รันชุดทดสอบตรวจสอบกฎ SQL Injection เฉพาะข้อผ่าน pytest
pytest -v --rule=REQUEST-942-APPLICATION-ATTACK-SQLI.yaml test/run.py

# 3. รันผ่าน CLI ของ FTW โดยตรง ระบุโฮสต์และพอร์ตเป้าหมาย
ftw run -d /path/to/coreruleset/tests/ -h 127.0.0.1 -p 8080
```

#### ตารางอธิบายพารามิเตอร์ของ OWASP FTW

| พารามิเตอร์ (Flag)         | ชนิดข้อมูล       | ความหมายและการทำงานเชิงลึก                                     | ตัวอย่างการใช้งานจริง                  |
|----------------------------|------------------|----------------------------------------------------------------|----------------------------------------|
| <code>-d, --dir</code>     | Folder Path      | ไดเรกทอรีที่บรรจุไฟล์สเปกทดสอบ YAML ของ OWASP CRS              | <code>-d ./tests/rule-tests/</code>    |
| <code>-h, --host</code>    | String (IP/Host) | IP Address หรือ Hostname ของ WAF Proxy ที่ต้องการทดสอบ         | <code>-h 127.0.0.1</code>              |
| <code>-p, --port</code>    | Integer          | พอร์ตที่ WAF เปิดรับการเชื่อมต่อ (เช่น 80, 443, 8080)          | <code>-p 8080</code>                   |
| <code>--rule</code>        | String / Pattern | ระบุชื่อไฟล์กฎหรือ Rule ID ที่ต้องการทดสอบแบบเฉพาะเจาะจง       | <code>--rule=*942*</code> (เฉพาะ SQLi) |
| <code>-v, --verbose</code> | Flag             | แสดงรายละเอียดการส่งคำขอและผลลัพธ์ของแต่ละ Test Case รายบรรทัด | <code>pytest -v</code>                 |

## 5. เครื่องมือที่ 3: WAFW00F

### Wafw00f — เครื่องมือระบุยี่ห้อและตรวจสอบการทำงานของ WAF (Fingerprinting)

ประเภท: Python CLI Tool | เหมาะสำหรับ: Penetration Tester, Security Auditor

**เครื่องมือนี้ทำอะไร:** ส่ง HTTP Requests หลากหลายลักษณะเพื่อกระตุ้นให้ WAF หน้าบ้านตอบสนอง จากนั้นนำ Header, Cookies และหน้า Error Page มาเทียบกับ Signature ฐานข้อมูลกว่า 150 ชนิด เพื่อระบุว่า มี WAF ขวางอยู่หรือไม่ และเป็นของผู้ให้บริการรายใด

**ช่วยอะไรในการทดสอบ WAF:** ใช้ยืนยันว่าการตั้งค่า Network และ Routing ถูกต้อง ทราฟฟิกวิ่งผ่าน WAF จริงก่อนเริ่มทำการทดสอบขั้นต่อไป

#### ชุดคำสั่งใช้งานจริง (Full Command Examples)

```
# 1. ตรวจสอบเป้าหมายแบบมาตรฐาน
wafw00f https://target-app.internal

# 2. ตรวจสอบแบบละเอียด และทดสอบ Signature ทุกยี่ห้อ (ไม่หยุดเมื่อเจอตัวแรก)
wafw00f https://target-app.internal -a -v

# 3. ตรวจสอบผ่าน HTTP Proxy (เช่น ส่งผ่าน Burp Suite เพื่อวิเคราะห์ Request)
wafw00f http://localhost:8080 -p http://127.0.0.1:8081
```

#### ตารางอธิบายพารามิเตอร์ของ Wafw00f

| พารามิเตอร์ (Flag)         | ชนิดข้อมูล   | ความหมายและการทำงานเชิงลึก                                                     | ตัวอย่างการใช้งานจริง                 |
|----------------------------|--------------|--------------------------------------------------------------------------------|---------------------------------------|
| <code>target</code>        | URL String   | เป้าหมายที่ต้องการตรวจสอบ (สามารถใส่เป็น Domain หรือ IP:Port ก็ได้)            | <code>https://example.com</code>      |
| <code>-a, --findall</code> | Flag         | ทดสอบตรวจสอบ Signature ของ WAF ทุกยี่ห้อที่มีในระบบ แทนที่จะหยุดเมื่อเจอตัวแรก | <code>wafw00f http://target -a</code> |
| <code>-v, --verbose</code> | Flag         | เพิ่มระดับการแสดงผลรายละเอียดการตอบสนองของเซิร์ฟเวอร์ (-v ถึง -vvv)            | <code>wafw00f http://target -v</code> |
| <code>-p, --proxy</code>   | String (URL) | กำหนด Proxy Server ให้ Wafw00f ส่งคำขอผ่าน (เช่น สำหรับการดักจับแพ็กเก็ต)      | <code>-p http://127.0.0.1:8080</code> |

| พารามิเตอร์ (Flag) | ชนิดข้อมูล | ความหมายและการทำงานเชิงลึก                                                      | ตัวอย่างการใช้งานจริง |
|--------------------|------------|---------------------------------------------------------------------------------|-----------------------|
| -t, --test         | String     | เจาะจงทดสอบเฉพาะ Signature ของยี่ห้อที่ระบุ (เช่น Cloudflare, ModSecurity, AWS) | -t ModSecurity        |

## 6. เครื่องมือที่ 4: OWASP ZAP (ZED ATTACK PROXY)

### OWASP ZAP — ทดสอบการกำบังช่องโหว่ของเว็บแอปพลิเคชัน (Compensating Control)

ประเภท: Full DAST Platform / Docker Scanner | เหมาะสำหรับ: AppSec, DevSecOps

**เครื่องมือนี้ทำอะไร:** เป็นเครื่องมือ Dynamic Application Security Testing (DAST) สำหรับค้นหาช่องโหว่ของเว็บแอปพลิเคชัน ในบริบทของการทดสอบ WAF วิศวกรจะนำ ZAP มาสแกนเว็บ 2 สภาวะ: (1) สแกนตรงสู่เซิร์ฟเวอร์หลังบ้านเพื่อหาช่องโหว่จริง และ (2) สแกนผ่าน WAF เพื่อดูว่า WAF สามารถกำบังช่องโหว่เหล่านั้นได้กี่เปอร์เซ็นต์

#### ชุดคำสั่งใช้งานจริงผ่าน Docker (Full Command Examples)

```
# รัน ZAP Baseline Scan ใน Docker เพื่อสแกนเว็บผ่าน WAF และบันทึกรายงานเป็น HTML และ JSON
docker run -v $(pwd):/zap/wrk/:rw -t zaproxy/zap-stable zap-baseline.py -t http://172.17.0.1:8080 -r zap_report.html -J zap_report.json -m 5 -I
```

ตารางอธิบายพารามิเตอร์ของ OWASP ZAP Baseline Scan

| พารามิเตอร์ (Flag) | ชนิดข้อมูล | ความหมายและการทำงานเชิงลึก                                                   | ตัวอย่างการใช้งานจริง     |
|--------------------|------------|------------------------------------------------------------------------------|---------------------------|
| -t <target>        | URL String | URL ของเว็บไซต์เป้าหมายที่ต้องการให้ ZAP ทำการ Crawl และสแกน                 | -t http://172.17.0.1:8080 |
| -r <report.html>   | File Name  | ชื่อไฟล์รายงานสรุปผลลัพธ์ในรูปแบบ HTML                                       | -r waf_audit_report.html  |
| -J <report.json>   | File Name  | ชื่อไฟล์รายงานสรุปผลลัพธ์ในรูปแบบ JSON สำหรับนำไปประมวลผลต่อ                 | -J zap_output.json        |
| -m <minutes>       | Integer    | ระยะเวลาที่กำหนดให้แมงมุม (Spider) ทำการสำรวจเส้นทางของเว็บแอป (เป็นนาที)    | -m 5 (จำกัด 5 นาที)       |
| -I                 | Flag       | ไม่ส่งรหัส Return Code เป็น Error เมื่อพบคำเตือนความเสี่ยง (Ignore Warnings) | -I                        |

## 7. เครื่องมือที่ 5: K6 (GRAFANA)

### k6 — วัดผลกระทบต่อความเร็ว (Added Latency P95/P99) และขีดจำกัด Throughput

ประเภท: Modern Load & Performance Testing CLI | เหมาะสำหรับ: Site Reliability Engineer (SRE), DevOps

**เครื่องมือนี้ทำอะไร:** เครื่องมือ Load Testing ประสิทธิภาพสูง เขียนคำสั่งควบคุมทราฟฟิกด้วย JavaScript ใช้ยิงจำลองผู้ใช้หลายร้อยคนพร้อมกัน เพื่อดูว่าเมื่อเปิดโหมด Deep Packet Inspection ของ WAF ระบบจะเพิ่มความหน่วง (Latency) กี่มิลลิวินาที

ตัวอย่างสคริปต์ทดสอบ k6 (ไฟล์ benchmark\_waf.js )

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 }, // ค่อยๆ เพิ่มผู้ใช้เป็น 20 คน
    { duration: '1m', target: 50 }, // ยังคงที่ที่ 50 ผู้ใช้พร้อมกัน
    { duration: '20s', target: 0 }, // ค่อยๆ ลดทราฟฟิก
  ],
  thresholds: {
    http_req_duration: ['p(95)<50'], // เกณฑ์ P95 Latency ต้องต่ำกว่า 50ms
    http_req_failed: ['rate<0.01'], // อัตรา Request ล้มเหลวต้องต่ำกว่า 1%
  },
};

export default function () {
  const res = http.get('http://localhost:8080/');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(0.5);
}
```

คำสั่งรันและตารางอธิบายพารามิเตอร์ของ k6

```
k6 run --summary-export=k6-summary.json benchmark_waf.js
```

| พารามิเตอร์ (Flag) | ชนิดข้อมูล  | ความหมายและการทำงานเชิงลึก                                                | ตัวอย่างการใช้งานจริง        |
|--------------------|-------------|---------------------------------------------------------------------------|------------------------------|
| run                | Command     | สั่งประมวลผลและรันสคริปต์ทดสอบโหลดตามเงื่อนไขที่กำหนดในไฟล์ .js           | k6 run test.js               |
| --vus              | Integer     | จำนวน Virtual Users ที่สร้างทราฟฟิกพร้อมกัน (Override ค่าในสคริปต์ได้)    | --vus 100 (จำลอง 100 ผู้ใช้) |
| --duration         | Time String | ระยะเวลาที่ยิงทดสอบอย่างต่อเนื่อง เช่น 30 วินาที หรือ 5 นาที              | --duration 2m                |
| --summary-export   | File Path   | ส่งออกข้อมูลสรุปทางสถิติ (P90, P95, P99, RPS, Drop rate) ออกเป็นไฟล์ JSON | --summary-export=result.json |

8. สรุปภาพรวมและแผนภูมิขั้นตอนการทดสอบ WAF (END-TO-END WORKFLOW)

เพื่อให้การประเมิน WAF เป็นไปอย่างเป็นระบบและครอบคลุมทุกมิติ ให้ปฏิบัติตามลำดับ 5 ขั้นตอนนี้:

- 1. **ขั้นตอนที่ 1: ตรวจสอบการไหลของทราฟฟิก (Routing Verification):**  
ใช้ wafw00f เพื่อยืนยันว่าคำขอวิ่งผ่าน WAF จริง ไม่ได้หลุดไปถึงเซิร์ฟเวอร์หลังบ้านตรงๆ
- 2. **ขั้นตอนที่ 2: รันการทดสอบภาพรวมและวัดระดับ Bypass (Automated Benchmark):**  
ใช้ GoTestWAF เพื่อสแกนครอบคลุม OWASP Top 10 + API และเทคนิคการหลบเลี่ยง (Encoders) เพื่อหาจุดอ่อนว่ามี Payload ใดที่หลุดรอด (Bypassed) บ้าง
- 3. **ขั้นตอนที่ 3: ตรวจสอบ False Positive ด้วยทราฟฟิกจริง (False Positive Audit):**  
ทดสอบยิงข้อมูลภาษาไทย, สระซ้อน, อักขระฟอร์มพิเศษ และข้อมูล JSON ขนาดใหญ่ เพื่อดูว่า WAF กีดกันผู้ใช้งานปกติหรือไม่
- 4. **ขั้นตอนที่ 4: ปรับจูนกฎที่ละข้อและป้องกันการถอยหลัง (Rule Tuning & Regression):**  
ใช้ OWASP FTW ตรวจสอบกฎรายข้อ และสร้างข้อยกเว้น (Exemption Rules) เฉพาะจุด
- 5. **ขั้นตอนที่ 5: วัดความหน่วงและขีดจำกัด Throughput (Performance Benchmarking):**  
ใช้ k6 ยิงโหลดเปรียบเทียบระหว่างสถานะ Direct Web Server (Baseline) กับการเปิด WAF เพื่อหาค่า Added Latency P95/P99

9. เกณฑ์มาตรฐานสำหรับการยอมรับเพื่อขึ้นสู่ระบบ PRODUCTION (GO-LIVE ACCEPTANCE CRITERIA)

| หมวดหมู่ตัวชี้วัด (Metric) | เกณฑ์มาตรฐานที่ยอมรับได้ | แนวทางการประเมินและการแก้ไข                                                      |
|----------------------------|--------------------------|----------------------------------------------------------------------------------|
| OWASP Core Detection Rate  | ≥ 85% - 90%              | ตรวจสอบผ่าน GoTestWAF หากต่ำกว่าเกณฑ์ต้องเปิดใช้ Rule Set เพิ่มเติม              |
| Bypass / Evasion Rate      | ≤ 15%                    | หาก Double URL หรือ Chunked Encoding หลุดรอด ต้องเปิดใช้ Recursive Normalization |

| หมวดหมู่ตัวชี้วัด (Metric)    | เกณฑ์มาตรฐานที่ยอมรับได้ | แนวทางการประเมินและการแก้ไข                                                      |
|-------------------------------|--------------------------|----------------------------------------------------------------------------------|
| False Positive Rate           | < 0.1%                   | คำขอปกติของผู้ใช้ต้องไม่โดน 403 Forbidden หากพบต้องเขียน Exemption Rule เฉพาะจุด |
| Added Latency (P95 Overhead)  | ≤ 15 - 25 ms             | ความหน่วงส่วนเพิ่มต้องไม่ทำให้ผู้ใช้รู้สึกช้าลงอย่างมีนัยสำคัญ วัดด้วย k6        |
| Emergency Fail-Open Mechanism | พร้อมใช้งาน 100%         | ต้องมีระบบ Bypass หรือสวิตช์ปิด WAF สำรองในกรณีฉุกเฉินเพื่อไม่ให้กระทบธุรกิจ     |